

Synchronous Replication in PostgreSQL

How to Convert Asynchronous PostgreSQL Replication to Synchronous Replication

Table of Contents

- **0. Overview**
 - **1. Environment**
 - **2. Goal**
 - **3. Verify Application Name on Primary**
 - **4. Edit Standby1 (rac1) Connection Info**
 - **5. Edit Standby2 (rac3) Connection Info**
 - **6. Verify Application Name on Primary (After Change)**
 - **7. Enable Synchronous Mode (Convert ASYNC to SYNC)**
 - **8. Validate Synchronization Settings**
 - **9. Summary of Results**
 - **10. Practical Test of Synchronous Replication**
 - **10.1 Check Current Status**
 - **10.2 Insert Test Data on Primary**
 - **10.3 Verify Data on Sync Standby**
 - **10.4 Test Sync Behavior (Stop Standby)**
 - **10.5 Observe Primary Blocking**
 - **10.6 Restart Standby**
 - **10.7 Observe Replication Lag**
-

0. Overview

- **Synchronous Streaming Replication (SYNC) in PostgreSQL ensures that a transaction is committed only after the data is successfully replicated to one or more standby servers.**
- **This provides zero data loss (RPO = 0) in case the primary crashes, because every committed transaction exists on at least one standby.**
- **PostgreSQL synchronous replication = Oracle Dataguard Maximum Protection mode**

How It Works

1. A client issues a COMMIT on the primary.
2. PostgreSQL writes the transaction to the WAL (Write-Ahead Log).
3. The WAL data is streamed to standby servers via the WAL sender (walsender).
4. WAL receiver (walreceiver) processes.

The primary waits until:

5. At least **one synchronous standby confirms** it has received the WAL record.
6. Only then does the primary report the transaction as committed to the client.

Advantages

- No data loss if the primary fails (RPO = 0).
- Ensures data durability across nodes.
- Suitable for financial or critical systems.

Disadvantages

- Higher latency for commits (client waits for standby acknowledgment).
- If synchronous standby is down or slow, the primary will pause commits.
- Requires stable, low-latency network.

1. Environment

NODE	IP	ROLE	SYNC TYPE	PURPOSE
rac1	192.168.2.21	Standby1	async	Convert ASYNC to SYNC
rac2	192.168.2.22	Primary	async	Main writer
rac3	192.168.3.20	Standby2	async	Will remain async

2. Goal

Convert ASYNC to SYNC only for Standby 1 (rac1)

rac1 → Standby 1 (Synchronous mode)
rac2 → Primary
rac3 → Standby 2 (Asynchronous mode)

Since rac1 operates in SYNC mode, the primary waits for commit acknowledgment from rac1 before completing each transaction.

This behavior ensures data consistency, as the primary confirms commits only after receiving acknowledgment from the synchronous standby.

If rac1 is stopped, the transaction (for example, process PID 6101) will

pause or hang until rac1 becomes available again – this is expected synchronous replication behavior.

Meanwhile, rac3 (the asynchronous standby) does not block transactions, since it doesn't participate in the synchronous commit process.

3. Verify Application Name on Primary

On Primary Node

Each standby must have a unique application_name in its connection info, but here both standby's have same application name.

```
[postgres@rac2 ~]$ psql -c "SELECT application_name, client_addr,
sync_state, sync_priority FROM pg_stat_replication;"
```

application_name	client_addr	sync_state	sync_priority
walreceiver	192.168.3.20	async	0
walreceiver	192.168.2.21	async	0

(2 rows)

```
[postgres@rac2 ~]$
```

4. Edit Standby1 (rac1) Connection Info

On Standby 1 (rac1)

```
[postgres@rac1 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
```

Do not edit this file manually!

It will be overwritten by the ALTER SYSTEM command.

```
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.22 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'rac1_standby_slot'
[postgres@rac1 ~]$
```

```
[postgres@rac1 ~]$ psql
```

```
psql (17.6)
```

```
Type "help" for help.
```

```

postgres=# ALTER SYSTEM SET primary_conninfo = 'user=repuser
password=repuser channel_binding=prefer host=192.168.2.22 port=5432
sslmode=prefer sslnegotiation=postgres sslcompression=0 sslcertmode=allow
sslsni=1 ssl_min_protocol_version=TLSv1.2 gssencmode=prefer
krbsrvname=postgres gssdelegation=0 target_session_attrs=any
load_balance_hosts=disable application_name=rac1';
ALTER SYSTEM
postgres=#

[postgres@rac1 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_slot_name = 'rac1_standby_slot'
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.22 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable
application_name=rac1'
[postgres@rac1 ~]$

[postgres@rac1 ~]$ psql -c "SELECT pg_reload_conf();"
 pg_reload_conf
-----
 t
(1 row)

[postgres@rac1 ~]$

```

5. Edit Standby2 (rac3) Connection Info

```

[postgres@rac3 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.22 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'rac3_standby_slot'
[postgres@rac3 ~]$
[postgres@rac3 ~]$ psql
psql (17.6)

```

Type "help" for help.

```
postgres=# ALTER SYSTEM SET primary_conninfo = 'user=repuser
password=repuser channel_binding=prefer host=192.168.2.22 port=5432
sslmode=prefer sslnegotiation=postgres sslcompression=0 sslcertmode=allow
sslsni=1 ssl_min_protocol_version=TLSv1.2 gssencmode=prefer
krbsrvname=postgres gssdelegation=0 target_session_attrs=any
load_balance_hosts=disable application_name=rac3';
ALTER SYSTEM
postgres=#
[postgres@rac3 ~]$
[postgres@rac3 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_slot_name = 'rac3_standby_slot'
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.22 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable
application_name=rac3'
[postgres@rac3 ~]$

[postgres@rac3 ~]$ psql -c "SELECT pg_reload_conf();"
pg_reload_conf
-----
t
(1 row)

[postgres@rac3 ~]$
```

6. Verify Application Name on Primary (After Change)

```
# On Primary node
[postgres@rac2 ~]$ psql -c "SELECT application_name, client_addr,
sync_state, sync_priority FROM pg_stat_replication;"
 application_name | client_addr | sync_state | sync_priority
-----+-----+-----+-----
 rac1             | 192.168.2.21 | async      |              0
 rac3             | 192.168.3.20 | async      |              0
(2 rows)

[postgres@rac2 ~]$
```

7. Enable Synchronous Mode (Convert ASYNC to SYNC)

On Primary (rac2):

examples:

```
/*
# ALTER SYSTEM SET synchronous_standby_names='application_name';
# ALTER SYSTEM SET synchronous_standby_names='(rac2, rac3)'; # both to be
synchronous
# ALTER SYSTEM SET synchronous_standby_names='ANY 1 (rac2, rac3)'; # only
one to be synchronous (any one of the two)
# ALTER SYSTEM SET synchronous_standby_names='*'; # all to be synchronous
*/
```

```
[postgres@rac2 ~]$ psql -c "SELECT application_name, client_addr,
sync_state, sync_priority FROM pg_stat_replication;"
```

application_name	client_addr	sync_state	sync_priority
rac1	192.168.2.21	async	0
rac3	192.168.3.20	async	0

(2 rows)

```
[postgres@rac2 ~]$ psql -c "ALTER SYSTEM SET
synchronous_standby_names='rac1';"
```

ALTER SYSTEM

```
[postgres@rac2 ~]$ psql -c "ALTER SYSTEM SET synchronous_commit = on;"
```

ALTER SYSTEM

```
[postgres@rac2 ~]$ psql -c "SELECT pg_reload_conf();"
```

pg_reload_conf

t <-----

(1 row)

```
[postgres@rac2 ~]$
```

8. Validate Synchronization settings

On Primary Node

```
[postgres@rac2 ~]$ psql -c "SHOW synchronous_standby_names;"
```

synchronous_standby_names

rac1 <----- This is Changed to SYNC.

(1 row)

```
[postgres@rac2 ~]$ psql -c "SHOW synchronous_commit;
synchronous_commit
-----
on <----
(1 row)

[postgres@rac2 ~]$
```

9. Summary of Results

```
[postgres@rac2 ~]$ psql -c "SELECT application_name, client_addr,
sync_state, sync_priority FROM pg_stat_replication;"
application_name | client_addr | sync_state | sync_priority
-----+-----+-----+-----
rac1             | 192.168.2.21 | sync      | 1
rac3             | 192.168.3.20 | async     | 0
(2 rows)

[postgres@rac2 ~]$

[postgres@rac2 ~]$ tail -f /pgData/pgsql17/data/log/postgresql-Sat.log
..
..
2025-10-18 05:18:17.188 EDT [1521] LOG:  parameter
"synchrounous_standby_names" changed to "rac1"
2025-10-18 05:18:26.727 EDT [4880] LOG:  standby "rac1" is now a
synchronous standby with priority 1
2025-10-18 05:18:26.727 EDT [4880] STATEMENT:  START_REPLICATION SLOT
"rac1_standby_slot" 0/1B000000 TIMELINE 2
```

10. Practical Test of Synchronous Replication

10.1 Check Current Status

```
[postgres@rac2 ~]$ psql -c "SELECT application_name, client_addr,
sync_state, sync_priority FROM pg_stat_replication;"
application_name | client_addr | sync_state | sync_priority
-----+-----+-----+-----
rac1             | 192.168.2.21 | sync      | 1
rac3             | 192.168.3.20 | async     | 0
(2 rows)
```

10.2 Insert Test Data on Primary

```
# On Primary :

[postgres@rac2 ~]$ psql
psql (17.6)
Type "help" for help.

postgres=#
postgres=# CREATE TABLE emp (name TEXT, designation TEXT, project TEXT,
company TEXT);
CREATE TABLE
postgres=# INSERT INTO emp VALUES ('Sugi', 'DBA', 'Jetstar', 'iGATE');
INSERT 0 1 <---- Got inserted
postgres=#

[postgres@rac2 ~]$ psql -c "SELECT * FROM EMP;"
  name | designation | project | company
-----+-----+-----+-----
  Sugi | DBA        | Jetstar | iGATE
(1 row)
```

[postgres@rac2 ~]\$

10.3 Verify Data on Sync Standby

```
# Standby 1 (rac1)

[postgres@rac1 ~]$ psql -c "SELECT * FROM EMP;"
  name | designation | project | company
-----+-----+-----+-----
  Sugi | DBA        | Jetstar | iGATE
(1 row)
```

[postgres@rac1 ~]\$

```
# Standby 1 (rac3)
[postgres@rac3 ~]$ psql -c "SELECT * FROM EMP;"
  name | designation | project | company
```



```
-----+-----+-----+-----
Sugi | DBA          | Jetstar | iGATE
(1 row)

[postgres@rac3 ~]$
```

10.4 Test Sync Behavior

On Standby 1 (rac1): shutdown

```
[root@rac1 ~]# systemctl stop postgresql-17.service
[root@rac1 ~]#
[root@rac1 ~]# ps -ef | grep postgres
root      4901    3426  0 12:56 pts/0    00:00:00 grep --color=auto
postgres
[root@rac1 ~]#
```

On Primary (rac2):

```
[postgres@rac2 ~]$ psql -c "INSERT INTO emp VALUES ('Teja', 'DBA', 'RCM',
'igATE');"
-- hanging, INSERT Waiting, not getting committed
```

```
# synchronous_standby_names='(rac1)', the insert will hang until rac1
(standby 1) is back online.
```

On Standby 2 (rac3):

```
[postgres@rac3 ~]$ psql -c "SELECT * FROM EMP;"
 name | designation | project | company
-----+-----+-----+-----
Sugi  | DBA         | Jetstar | iGATE
Teja  | DBA         | RCM     | iGATE <--- i can see data on
ASYNC Standby server (rac3)
(2 rows)

[postgres@rac3 ~]$
```

10.5 Observe Primary Blocking

Taken another session of Primary (rac2) and run as below.

```
[postgres@rac2 ~]$ psql -c "SELECT pid, state, wait_event_type, wait_event
FROM pg_stat_activity;"
```

pid	state	wait_event_type	wait_event
5620	active	IPC	SyncRep
5645	active		
5062		Activity	AutovacuumMain
5064		Activity	LogicalLauncherMain
5066	active	Activity	WalSenderMain
5058		Activity	CheckpointerMain
5059		Activity	BgwriterHibernate
5061		Activity	WalWriterMain
5063		Activity	ArchiverMain

(9 rows)

```
[postgres@rac2 ~]$
```

```
[postgres@rac2 ~]$ psql -c "SELECT pid, username, application_name,
client_addr, client_hostname, state, wait_event_type, wait_event FROM
pg_stat_activity WHERE state = 'active';"
```

pid	username	application_name	client_addr	client_hostname	state	wait_event_type	wait_event
5656	postgres	psql					
	active						
5620	postgres	psql					
	active	IPC				SyncRep	
5066	repuser	rac3	192.168.3.20	rac3			
	active	Activity				WalSenderMain	

(3 rows)

```
[postgres@rac2 ~]$
```

10.6 Restart Standby

```
[root@rac1 ~]# systemctl start postgresql-17.service
[root@rac1 ~]#
```

10.7 Observe Replication Lag

After restart postgres on standby 1 (rac1), INSERT statement got completed on primary.

```
[postgres@rac2 ~]$ psql -c "INSERT INTO emp VALUES ('Teja', 'DBA', 'RCM',
'iGATE');"
INSERT 0 1
[postgres@rac2 ~]$
```

Verify lag from primary (rac2).

```
[postgres@rac2 ~]$ psql
psql (17.6)
Type "help" for help.
```

```
postgres=# SELECT application_name, client_addr, sync_state, sent_lsn,
write_lsn, flush_lsn, replay_lsn
```

```
postgres-# FROM pg_stat_replication;
```

application_name	client_addr	sync_state	sent_lsn	write_lsn	flush_lsn	replay_lsn
rac3	192.168.3.20	async	0/26F8CF18	0/26F8CF18	0/26F8CF18	0/26F8CF18
rac1	192.168.2.21	sync	0/26F8CF18	0/26F8CF18	0/26F8CF18	0/26F8CF18

(2 rows)

```
postgres=#
```

```
[postgres@rac2 ~]$ psql -c "
```

```
> SELECT
```

```
> application_name AS standby_name,
```

```
> client_addr AS standby_ip,
```

```
> state,
```

```
> sync_state,
```

```
> pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS byte_lag
```

```
> FROM pg_stat_replication;
```

```
> "
```

standby_name	standby_ip	state	sync_state	byte_lag
--------------	------------	-------	------------	----------

```
-----+-----+-----+-----+-----
rac3      | 192.168.3.20 | streaming | async   |          0 <---
rac1      | 192.168.2.21 | streaming | sync    |          0 <---
(2 rows)
```

[postgres@rac2 ~]\$

Verify Data

On Primary (rac2):

```
[postgres@rac2 ~]$ psql -c "SELECT * FROM EMP;"
 name | designation | project | company
-----+-----+-----+-----
Sugi  | DBA         | Jetstar | iGATE
Teja  | DBA         | RCM     | iGATE
(2 rows)
```

[postgres@rac2 ~]\$

On Standby (rac1) 1:

```
[postgres@rac1 ~]$ psql -c "SELECT * FROM EMP;"
 name | designation | project | company
-----+-----+-----+-----
Sugi  | DBA         | Jetstar | iGATE
Teja  | DBA         | RCM     | iGATE
(2 rows)
```

[postgres@rac1 ~]\$

On Standby 2 (rac3):

```
[postgres@rac3 ~]$ psql -c "SELECT * FROM EMP;"
 name | designation | project | company
-----+-----+-----+-----
Sugi  | DBA         | Jetstar | iGATE
Teja  | DBA         | RCM     | iGATE
(2 rows)
```

```
[postgres@rac3 ~]$
```

Caution: *Your use of any information or materials on this website is entirely at your own risk. It is provided for educational purposes only. It has been tested internally, however, we do not guarantee that it will work for you. Ensure that you run it in your test environment before using.*

Thank you,

Rajasekhar Amudala

Email: br8dba@gmail.com

Linkedin: <https://www.linkedin.com/in/rajasekhar-amudala/>